Task Behavior Graphs: A Domain-Driven Model for Profiling Queue Worker Tasks under Uncertain Resource Demand

Anonymous Author(s)

Abstract

TODO

Keywords: queue workers; task profiling; workload characterization; resource demand; worker occupancy; behavior graphs; observability; instrumentation; admission control; overload regulation.

1 Introduction

Queue-based execution is widely used to decouple work production from work execution. A producer submits a task instance to a queue, and one or more workers later reserve, execute, retry, acknowledge, or fail that work. Although this pattern is often presented as a background-job abstraction, production queues frequently execute heterogeneous and operationally significant work: replication, repair, cache rebuilds, imports, media processing, external service calls, validation, notification delivery, and other asynchronous operations. In such systems, a worker is not merely a message consumer. It is an execution boundary in which logical task attempts consume active resources, hold bounded execution capacity, and affect downstream control decisions.

This paper studies profiling at the level of logical task attempts inside queue-worker systems. This level is important because the measurements that are easier to collect are usually coarser than the behavior that operators need to understand. Operating systems and container runtimes expose process-, cgroup-, container-, or node-level counters. Queue frameworks expose aggregate backlog, latency, failure, and retry metrics. Observability systems expose metrics, traces, logs, and runtime profiles. These observations provide evidence, but they do not by themselves identify which task kind or attempt created pressure, which task features were relevant, or which execution-context factors changed the behavior of that attempt.

A central distinction in this paper is between conditions, costs, and consequences. Execution context denotes the surrounding state under which an attempt executes: worker state, node pressure, dependency health, network condition, disk pressure, cache state, retry state, tenant pressure, or other domain-specific state. Resource demand and worker occupancy are different: they are cost-side outputs attributed to the attempt. Resource demand describes active resource need, such as CPU time or network transfer. Worker occupancy describes bounded execution capacity held over time, such as worker-slot time, lease time, connection hold time, or lock hold time. Operational risk is a downstream consequence of underestimating, misclassifying, or mishandling an attempt; it is not itself the execution cost.

The central difficulty is that task cost is conditional rather than purely intrinsic. A task kind and a payload-size bucket may provide a useful baseline, but they do not fully determine resource demand or worker occupancy. For example, two replication attempts with similar payloads may behave differently when one runs against a healthy replica set and another runs against degraded peers, unstable network paths, high retry rates, or disk pressure. These conditions are execution-context factors. Their effect may be observed as higher dependency wait time, longer

worker-slot time, more retries, or higher residual error. The context and the observed cost are related, but they are not the same quantity.

This distinction changes the profiling problem. The system should not ask only which telemetry streams can be collected. It must decide which observations are semantically relevant for a task kind, how raw observations become model inputs, which quantities describe active resource demand, which quantities describe worker occupancy, which observations are symptoms or outcomes, and when additional instrumentation is worth its overhead. A task that consumes little CPU but holds a worker slot or dependency connection for a long time may be more damaging to queue throughput than a short compute-dominant task. Conversely, collecting detailed traces for every stable, low-risk task can add overhead without improving the decisions that matter.

Existing resource-management and workload-analysis work motivates parts of this view. Cluster managers reason about multiple resource dimensions and observed usage [6,12]; workload studies show heterogeneity, dynamic behavior, and task-usage variation at scale [1, 10, 13]; workload classification systems infer behavior rather than relying only on static resource assumptions [4, 5]; and large distributed systems are often shaped by tail behavior rather than average case alone [2]. Observability systems provide essential traces, metrics, and profiles, but they also introduce overhead and require sampling or other budgeted policies [7, 8, 11]. These lines of work do not remove the need for a queue-worker-specific model that connects logical tasks, execution context, worker occupancy, uncertainty, and profiling policy.

This paper proposes *Task Behavior Graphs* as such a model. A Task Behavior Graph is associated with a task kind or task family. Its nodes represent typed features, demand dimensions, occupancy dimensions, uncertainty indicators, and risk-relevant outputs. Its edges encode explicit influence assumptions such as baseline demand, contextual amplification, additive overhead, symptom links, outcome links, and risk propagation. The graph is not treated as proof of causality. Instead, it makes the model assumptions visible so that they can be validated, calibrated, revised, or marked as uncertain.

The paper makes five contributions. First, it frames queue-worker task profiling as conditional behavior estimation rather than raw telemetry collection. Second, it separates intrinsic task features, execution context, active resource demand, worker occupancy, uncertainty, and operational risk into distinct model roles. Third, it introduces Task Behavior Graphs as a task-kind- or task-family-level representation for connecting these roles. Fourth, it describes how raw metrics, traces, logs, profiles, and domain events can be mapped into canonical features through descriptors, manifests, and domain-specific extensions. Fifth, it outlines how profiling policy can be selected from task behavior, uncertainty, consequence, and instrumentation budget.

The remainder of the paper is organized as follows. Section 2 defines the core terminology and notation. Section 3 provides the technical background and motivation. Section 4 states the profiling problem. Sections 5–8 introduce the cost model, influence domains, taxonomy, and graph representation. Sections 9–13 discuss feature mapping, profiling methods, observation points, uncertainty, risk, and policy selection. Section 14 describes manifests and domain packs, and Section 15 applies the model to representative task kinds. The final sections discuss evaluation, limitations, related work, and conclusions.

2 Core Terminology and Notation

This section defines only the core terms required to state the problem and introduce the model. The paper uses a larger terminology set, but the complete reference glossary is deferred to Appendix A so that the main text can proceed from the problem statement to the model without front-loading implementation and lifecycle details.

The terminology is role-specific. Task identity and intrinsic task features describe what work is requested. Execution context describes the environment and state under which an

attempt executes. Resource demand and worker occupancy are cost-side outputs attributed to the attempt. Observations provide evidence; canonical features are typed model inputs derived from observations or metadata. Uncertainty and operational risk are downstream interpretations of estimates, residuals, missing information, and consequences. These roles are kept distinct because the same operational event can connect several roles without making them synonyms. For example, degraded dependency health is an execution-context factor; dependency wait time is an observed symptom or intermediate duration; connection hold time and worker-slot time are occupancy dimensions; and queue starvation is a possible operational risk.

2.1 Core Terms

Table 1: Core terms used in the main model.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue. A task kind describes what work is requested, but it does not fully determine resource demand, worker occupancy, or risk under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource usage, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
task behavior	The observed or estimated execution pattern of a task attempt or task kind under a given execution context, including resource demand, worker occupancy, outcome, uncertainty, and risk-relevant effects.
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.

Term	Definition
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
influence domain	A semantic role assigned to a feature, observation, output, or model concept. Influence domains separate task identity, intrinsic demand, execution context, resource demand, worker occupancy, outcome, uncertainty, risk, and profiling observations.
resource demand	The expected or required active resource need of a task attempt along a resource dimension, such as CPU time, memory peak, disk I/O, network I/O, runtime allocation, or accelerator usage. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.
worker occupancy	Bounded execution capacity held by a task attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant-level concurrency time.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension in queue-worker systems.
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is modeled separately from task cost.
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph makes modeled influence assumptions explicit; it is not assumed to prove causality by itself.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context. A profiling policy may choose minimal accounting, detailed accounting, tracing, sampling, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution.

2.2 Notation

Table 2: Notation used throughout the paper.

Symbol	Meaning
$\mathcal{K}$	The set of task kinds.
$\mathcal{I}$	The set of task instances.
$\mathcal{A}$	The set of task attempts.
$i \in \mathcal{I}$	A task instance.
$a \in \mathcal{A}$	A task attempt.
$k(i) \in \mathcal{K}$	The task kind of task instance i.
$i(a) \in \mathcal{I}$	The task instance executed by attempt a.
$\mathcal{D}$	The set of influence domains.
$\mathcal{F}$	The set of canonical features.
z_a	The canonical feature vector available for attempt a after feature mapping and normalization.
$\mathcal{R}$	The set of resource-demand dimensions.
$\mathcal{O}$	The set of worker-occupancy dimensions.
$D_r(a)$	Observed resource demand or usage for resource dimension $r \in \mathcal{R}$ during attempt a.
$\widehat{D}_r(a)$	Estimated resource demand for resource dimension $r \in \mathcal{R}$ during attempt a.
$U_o(a)$	Observed worker occupancy for occupancy dimension $o \in \mathcal{O}$ during attempt a.
$\widehat{U}_o(a)$	Estimated worker occupancy for occupancy dimension $o \in \mathcal{O}$ during attempt a.
$\epsilon_r(a)$	Residual error for resource-demand dimension r, defined as the part of observed resource behavior not explained by the estimate.
$\epsilon_o(a)$	Residual error for worker-occupancy dimension o, defined as the part of observed occupancy behavior not explained by the estimate.
$\mathrm{Risk}(a)$	The operational-risk estimate for attempt a.
$\mathcal{G}_k$	The Task Behavior Graph associated with task kind k.
V_k	The set of nodes in Task Behavior Graph $\mathcal{G}_k$.
E_k	The set of edges in Task Behavior Graph $\mathcal{G}_k$.
$v \in V_k$	A node in Task Behavior Graph $\mathcal{G}_k$.
$e \in E_k$	An edge in Task Behavior Graph $\mathcal{G}_k$.
$\delta(v)$	The influence domain assigned to graph node v.

Symbol	Meaning
w_e	The estimated sensitivity or weight associated with graph edge e.
γ_e	The confidence value associated with graph edge e.

3 Background and Motivation

This section provides the technical background needed for the problem statement. It does not introduce the full model. Instead, it explains why queue-worker task profiling cannot be reduced to process-level monitoring, queue-level metrics, or generic tracing. The section first identifies the task attempt as the relevant execution unit. It then separates three roles that are often conflated: attempt-side resource demand, attempt-side worker occupancy, and environment-side execution-context pressure. Finally, it motivates conditional behavior, semantic feature mapping, and budgeted profiling.

3.1 Queue-Worker Execution and Task Attempts

Queue-worker systems execute work through a lifecycle that usually includes enqueue, reserve or dequeue, start, one or more processing stages, retry or backoff when needed, commit or output write, and acknowledgement or failure. A single task instance can produce multiple attempts because of retry policy, lease expiration, timeout, cancellation, worker failure, or rescheduling. For profiling, this matters because each attempt may execute under a different worker state, dependency state, retry state, or domain state.

The attempt is therefore the most precise unit for attributing observed behavior. A task kind describes the semantic type of requested work, and a task instance describes one queued unit of that kind. The attempt describes one concrete execution under one execution context. Resource usage, worker occupancy, outcome, and residual error are usually measured or estimated most cleanly at this attempt boundary. Aggregating only by queue, worker process, or node can reveal pressure, but it can hide which task kind, attempt, or context produced the observed behavior.

A worker may hold capacities that are not visible as high active utilization. During an attempt it may hold a worker slot, task lease, dependency connection, lock, semaphore permit, or tenant-level concurrency slot. These held capacities are worker-occupancy outputs of the attempt. They should be distinguished from the surrounding worker or worker-pool pressure. For example, a worker pool may already be 90% occupied before an attempt starts; that pool state is execution context. If the attempt then holds one worker slot for 30 seconds, that duration is worker-slot occupancy attributed to the attempt.

3.2 Resource Demand, Occupancy, and Pressure

Distributed resource-management systems commonly reason about multiple resource dimensions rather than a single scalar load. Cluster managers and allocation policies account for dimensions such as CPU, memory, storage, and other capacity constraints when placing, admitting, or allocating work [6, 12]. Queue-worker profiling inherits this multi-resource setting, but adds an important distinction: active resource demand, worker occupancy, and execution-context pressure are different roles.

Active resource demand describes resources consumed by the attempt itself, such as CPU time, memory peak, allocation volume, disk reads and writes, network transfer volume, or accelerator use. Worker occupancy describes bounded execution capacity held over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or

tenant concurrency time. Execution-context pressure describes the surrounding state that may influence the attempt, such as CPU saturation, memory pressure, disk queue depth, network instability, dependency saturation, connection-pool pressure, or worker-pool utilization.

These roles are related but not interchangeable. A compute-dominant attempt may consume CPU intensely for a short interval. A dependency-bound attempt may consume little CPU while holding a worker slot and a connection for much longer. A degraded dependency may increase dependency wait time, which may in turn increase worker-slot time if the slot remains held during the wait. In that chain, dependency health is a context factor, dependency wait time is a symptom or intermediate duration, and worker-slot time is an occupancy cost. Treating all three as the same quantity would obscure both attribution and control semantics.

Demand must also be separated from pressure. Demand is a property estimated or observed for the attempt. Pressure is a property of the execution context. CPU demand is not CPU pressure; disk I/O demand is not disk pressure; network transfer volume is not network instability; and worker-slot time is not worker-pool pressure. Confusing these terms makes it unclear whether an attempt is intrinsically expensive, contextually amplified, or merely observed under degraded conditions. A profiling model should therefore preserve demand dimensions, occupancy dimensions, and context-pressure features before projecting them into any policy-specific score.

3.3 Workload Variability and Context Dependence

Production workloads are heterogeneous and dynamic. Prior studies of large compute clusters and co-located workloads show substantial variation in task usage, infrastructure conditions, and workload behavior across time, machines, and task populations [1, 10, 13]. Workload classification systems such as Paragon and Quasar similarly motivate inferring behavior from observed properties and performance constraints rather than relying only on static declarations or user-provided reservations [4,5]. For queue workers, the implication is direct: task kind, queue name, and declared size can provide a baseline, but not a complete behavior model.

A replication task illustrates the issue. Two attempts may have the same task kind and similar payload size, but execute against different replica sets. Under stable network conditions and healthy peers, the attempt may complete near its baseline. Under degraded peer health, packet loss, retransmissions, high replication lag, disk pressure, or retry backoff, the same logical operation can produce larger worker-slot time and a different risk profile. The task did not become a different task kind. Its execution context changed, and that context amplified some cost dimensions.

The same pattern appears in other task families. A transformation task may be mostly shaped by record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. An external-service task may be shaped by dependency health, fan-out width, timeout policy, connection-pool availability, and retry state. A repair, compaction, cache rebuild, or migration task may be shaped by hidden domain state such as fragmentation, dirty data volume, replica divergence, cache warmth, or disk queue depth. A useful profiling model must therefore ask which features are relevant for a task kind rather than impose one global metric set on all tasks.

Average behavior is not enough. Large-scale distributed systems are often shaped by tail behavior and stragglers [2, 3]. In a queue-worker system, rare attempts that hold worker slots, leases, locks, or dependency connections for unusually long periods can reduce effective capacity and increase queueing delay even when mean duration appears acceptable. This motivates estimating not only central tendency but also uncertainty, residual error, drift, and high-percentile behavior when those signals affect control decisions.

3.4 Observability Signals and Feature Mapping

Metrics, traces, logs, runtime profiles, and domain events provide evidence about execution. Distributed tracing systems such as Dapper and modern telemetry systems expose spans, metrics, and runtime observations that help engineers understand distributed behavior [9, 11]. However, observations are not a profiling model by themselves. A metric, span, or log record does not intrinsically say whether it is a task feature, a context factor, a resource measurement, an occupancy measurement, a symptom, an outcome, or a risk indicator.

This distinction prevents double counting and unclear attribution. Network degradation may contribute to dependency wait, retry count, wall-clock duration, and worker-slot occupancy. These observations are connected, but they do not have the same role. Network degradation is a context factor. Retry count is an outcome or symptom of repeated execution. Wall-clock duration is elapsed time between boundaries. Worker-slot time is occupancy only if the attempt holds a worker slot during that elapsed interval. Treating all of them as independent causes can count one underlying degradation multiple times. Treating only the final duration as cost has the opposite problem: it hides the mechanism that produced the delay.

Feature mapping is the layer that provides semantic interpretation. Raw observations such as packet loss, replica lag, connection-pool wait time, timeout rate, or allocation volume must be normalized and mapped into canonical features before they can feed a reusable model. Different domains may expose different raw signals for similar influences, and similar metric names may carry different meanings in different systems. A model that operates over canonical features can separate domain integration from behavior evaluation.

3.5 Profiling Overhead and Observation Policy

Observation has cost. Instrumentation, tracing, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export can add CPU work, allocations, memory pressure, storage volume, and network traffic. Dapper was designed with low overhead as a central concern, and OpenTelemetry treats sampling and telemetry performance as operational constraints [7,8,11]. Therefore, collecting more observations is not automatically better.

The right observation strategy depends on the task kind and on the value of the information being collected. Stable, low-risk tasks may need only minimal accounting. Unknown, high-tail, context-sensitive, or high-consequence tasks may justify detailed accounting, sampled tracing, tail-aware capture, or diagnostic profiling. The profiling policy must also degrade safely under overload: the instrumentation used to understand a system should not become a new source of unbounded work or cardinality.

This background motivates a decision-oriented view of profiling. The goal is not to maximize telemetry volume. The goal is to collect the cheapest observations that improve estimates of resource demand, worker occupancy, uncertainty, and operational risk enough to support better control decisions.

3.6 Motivating Gap

The preceding points expose a gap between available observations and the model needed for queue-worker task profiling. Resource-management systems reason about multiple resources; workload studies show variability and context dependence; observability systems expose metrics, traces, logs, and profiles. Yet these pieces do not by themselves specify how to attribute behavior to logical task attempts, separate intrinsic demand from execution context, account for worker occupancy, distinguish causes from symptoms, or choose instrumentation under an overhead budget.

The gap becomes clearer when tasks are compositional. A single task kind may include CPU processing, network transfer, dependency calls, retries, commits, and cleanup. Some quantities

compose by summation, others by maximum, critical path, retry expansion, or occupancy-specific rules. Without an explicit model of metric roles and composition semantics, the system cannot reliably combine component behavior into an attempt-level estimate.

The next section formulates this gap as a task-profiling problem. Later sections introduce the cost model, influence domains, task taxonomy, Task Behavior Graphs, feature mapping, and profiling-policy selection as a structured response to that problem.

4 Problem Statement

The preceding section motivates a gap between available observations and the model needed to profile logical task attempts in queue-worker systems. This section states the problem addressed by the paper. It defines the system model, the information available to the profiler, the outputs expected from the profiling model, the main constraints, and the non-goals. The purpose is to separate the problem from the model proposed in later sections.

At a high level, the problem is to estimate the behavior of a task attempt from its task kind, task-instance features, execution-context observations, and available history, while respecting profiling overhead. The estimate must not be reduced to a single scalar cost. It must preserve active resource demand, worker occupancy, uncertainty, and operational risk as distinct outputs because these quantities support different operational decisions. Throughout this section, execution context denotes input-side environmental state; resource demand and worker occupancy denote cost-side outputs attributed to the attempt; and operational risk denotes downstream consequence.

4.1 System Model

We consider queue-worker systems in which producers submit task instances to one or more queues and workers execute task attempts. A task kind identifies the semantic type of work requested. A task instance is one queued unit of that kind. A task attempt is one concrete execution of a task instance. A single task instance may produce multiple attempts because of retries, lease expiration, timeouts, cancellation, worker failure, or rescheduling.

The profiling boundary is the task attempt. This boundary is narrower than a worker process, container, node, or worker pool, but broader than a single function call or trace span. An attempt may include multiple phases, such as input fetching, parsing, computation, dependency calls, retries, output writes, commits, acknowledgement, and cleanup. These phases may be sequential, parallel, conditional, retried, or partially overlapping. The paper does not assume that every phase is perfectly instrumented.

Each attempt executes under an execution context. The context is not the cost of the attempt. It is the surrounding state that may influence the cost. The context may include worker state, worker-pool pressure, node pressure, dependency health, network condition, disk pressure, cache state, replica state, tenant pressure, retry state, and domain-specific hidden state. Some context factors are directly observable, some are derived from raw observations, and some remain hidden. Hidden or missing factors are represented indirectly through residual error, uncertainty, and reduced confidence.

The model is task-kind-specific. Different task kinds may require different features, different observation points, and different profiling policies. A replication task kind may be sensitive to network condition, replica health, replication lag, disk pressure, and retry state. A compute-oriented transform may be more sensitive to record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. The system therefore cannot assume one global feature set or one global metric list that is equally relevant to all task kinds.

4.2 Available Information

For an attempt a, the profiler may have access to four classes of information. First, task identity describes what is being executed. It may include the task kind, task version, queue, producer, tenant, tenant class, priority, deadline, and attempt number. Task identity is necessary for selecting a task-kind-specific model, but it is not by itself a cost estimate.

Second, intrinsic task features describe work contained in the task instance. Examples include payload size, record count, object count, partition size, batch size, input format, operation mode, declared resource hints, or domain metadata. These features may be available before execution and can provide a baseline for expected work. Their presence and precision are domain-specific.

Third, execution-context observations describe the environment in which the attempt is executed or is expected to execute. Examples include worker-pool pressure, node CPU or memory pressure, disk queue depth, network instability, dependency health, connection-pool pressure, cache warmth, replica health, replication lag, tenant quota usage, and retry state. These observations may be fresh, stale, missing, aggregated, or only indirectly related to the attempt. They are input-side conditions, not worker-occupancy values. For example, connection-pool pressure is context; connection hold time attributed to the attempt is occupancy.

Fourth, historical observations describe previous attempts of the same or related task kinds. They may include observed resource usage, worker occupancy, durations, retry counts, outcomes, residual errors, tail quantiles, or drift indicators. Historical data may be unavailable for new task kinds, new task versions, rare contexts, or recently changed workloads. The problem must therefore support both calibrated and cold-start cases.

Raw observations are not assumed to be model-ready. Metrics, logs, traces, runtime profiles, and domain events must first be interpreted, normalized, and mapped into canonical features. This mapping is part of the profiling problem because the same raw metric can play different roles in different domains, and similar operational influences can be exposed through different raw metrics.

4.3 Desired Outputs

The profiler should produce a structured estimate for a task attempt or for a class of comparable attempts. The first output is an estimate of active resource demand. For each relevant resource dimension $r \in \mathcal{R}$, the model should estimate $\widehat{D}_r(a)$ or a distribution over possible values. Relevant dimensions may include CPU time, memory peak, disk reads, disk writes, network input, network output, allocation volume, or accelerator usage.

The second output is an estimate of worker occupancy. For each relevant occupancy dimension $o \in \mathcal{O}$, the model should estimate $\widehat{U}_o(a)$ or a distribution over possible values. Relevant dimensions may include worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, and tenant concurrency time. Occupancy is modeled separately because an attempt can hold scarce execution capacity without consuming much active CPU or memory. Occupancy is also separate from execution-context pressure: worker-pool pressure may influence an attempt, while worker-slot time is capacity held by that attempt.

The third output is uncertainty and confidence. The model should indicate how reliable its estimates are, whether residual error is expected to be high, whether behavior has drifted, whether historical samples are insufficient, and whether tail behavior is operationally significant. The paper does not require a single universal uncertainty metric. Later sections may use residuals, confidence values, quantile ratios, drift indicators, or other measures where appropriate.

The fourth output is operational risk. Risk is not the same as task cost. It captures the consequence of underestimating, misclassifying, or mishandling an attempt. Relevant risks may include queue starvation, retry amplification, SLO violation, replication lag, tenant impact, dependency overload, and cascading overload. Risk depends not only on estimated demand and occupancy but also on the blast radius and the operational meaning of the task kind.

The fifth output is a profiling-policy recommendation. Given the task kind, context, uncertainty, risk, and profiling budget, the system should determine what observation strategy is justified. Possible strategies include minimal accounting, detailed accounting, sampled tracing, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution. The policy output is a profiling decision, not a final scheduling or routing decision.

4.4 Core Challenges

The first challenge is conditional behavior. Attempts with the same task kind and similar intrinsic features may behave differently under different execution contexts. A model based only on task kind, queue, or payload bucket can provide a baseline, but it cannot explain context-dependent amplification, suppression, or drift.

The second challenge is multidimensionality. Active resource demand, bounded worker occupancy, execution-context pressure, uncertainty, and operational risk have different meanings and units. Combining them prematurely into one scalar hides information needed by downstream control layers. A scalar score may be a useful projection, but it should be derived from a structured estimate rather than replace it.

The third challenge is attribution. Process-level, container-level, node-level, and queue-level observations are often easier to collect than task-attempt-level observations. These aggregate observations may indicate pressure, but they do not automatically explain which task kind, instance, attempt, phase, context factor, or dependency produced it.

The fourth challenge is semantic interpretation. Raw observations do not state whether they represent intrinsic work, execution context, active resource usage, worker occupancy, a symptom, an outcome, an uncertainty indicator, or a risk indicator. Without explicit semantic roles, a model may double-count related signals. For example, network instability, retry count, wall-clock duration, and worker-slot time may all reflect one degraded dependency path, but they should not be treated as independent causes. Network instability is context; retry count and wall-clock duration are symptoms or outcomes; worker-slot time is occupancy if the slot remains held.

The fifth challenge is compositional behavior. A task kind may include multiple behavior components, such as CPU processing, network transfer, dependency calls, retries, commits, and cleanup. Some quantities compose by summation, some by maximum, some by critical path, and some by retry expansion or occupancy-specific rules. The profiling problem must therefore preserve enough structure to combine observations consistently.

The sixth challenge is partial observability. Important influences may be missing, stale, hidden, or domain-specific. The model cannot assume that all relevant state is observable. Instead, unexplained behavior should remain visible through residual error, reduced confidence, or a higher profiling level.

The seventh challenge is profiling overhead. Instrumentation is not free. Detailed tracing, runtime profiling, high-cardinality labels, frequent sampling, and telemetry export can affect the workload being measured. A profiling model must therefore decide not only what would be useful to observe, but what is worth observing under an overhead budget.

4.5 Scope and Assumptions

This paper assumes that task attempts can be identified at least by task kind and attempt boundary. The model is less useful when all work is invisible inside one undifferentiated worker loop. The paper does not require perfect instrumentation, but it assumes that minimal accounting is possible for at least some attempts, including start, finish, outcome, and task identity.

The paper assumes that some task metadata or intrinsic features may be available before or during execution. The exact features are domain-specific. A small validation task, a replication task, an external API fan-out task, and a media transformation task need not expose the same feature set.

The paper assumes that execution-context observations may be available but incomplete. These observations may come from workers, nodes, queues, dependencies, telemetry systems, domain services, or historical profiles. The model must tolerate missing, stale, noisy, or partially aggregated context.

The paper assumes that profiling decisions are useful even when final scheduling, routing, placement, or admission decisions are made by another layer. The profiling model supplies structured estimates and observation policies. Downstream control layers may consume these outputs, but their full design is outside the scope of the paper.

The paper also assumes an open-world metric environment. No core system can know all possible domain-specific metrics in advance. The model must therefore allow raw observations to be mapped into canonical features and must keep missing or unexplained influences explicit instead of hiding them behind a fixed global metric list.

4.6 Non-Goals

This paper does not propose a complete distributed scheduler. It does not define a placement algorithm, routing algorithm, gossip protocol, load-balancing policy, or cluster-wide resource allocator. Such systems may use the estimates produced by task profiling, but they are not the object of this paper.

This paper does not replace metrics, logs, traces, runtime profiles, or existing observability tools. These systems provide observations. The problem addressed here is how to interpret observations for task-kind-specific profiling and how to decide which observations are justified.

This paper does not claim that Task Behavior Graphs prove causality. A graph can make modeled influence assumptions explicit, but causal validation requires additional evidence, experiments, interventions, or domain knowledge. The graph is a profiling model, not a causal proof.

This paper does not attempt to define a globally complete taxonomy of all workload metrics. It assumes that new domains can introduce new observations, features, and behavior components. Completeness is task-kind- and domain-specific rather than global.

This paper does not require exact prediction for arbitrary tasks. The goal is to produce structured estimates, expose uncertainty, and guide profiling policy. High residual error, drift, missing features, or high tail risk are valid model outputs, not merely failures to hide.

Finally, this paper does not specify a full production implementation of a package manager, expression language, runtime virtual machine, or cross-language manifest compiler. Later sections discuss manifests and implementation considerations only to the extent needed to make the proposed model concrete.

5 Task Cost Model

Section 4 states that task profiling should produce structured estimates rather than a single scalar load value. This section defines the cost-side part of that estimate. In this paper, task cost denotes active resource demand, bounded worker occupancy, and execution overhead associated with a task attempt. Operational risk is not part of task cost; it is modeled later as the consequence of underestimating, misclassifying, or mishandling an attempt.

The purpose of the cost model is to make three distinctions explicit. First, cost is evaluated at attempt granularity, even when the reusable model structure is attached to a task kind. Second, active resource demand and worker occupancy are different families of quantities and must be estimated separately. Third, observed usage may differ from the estimate; the unexplained difference remains visible as residual error rather than being hidden by the model.

5.1 Cost as an Attempt-Level Quantity

Section 4 identified active resource demand and worker occupancy as required outputs of task profiling. This section defines the cost model used to represent these outputs. The model is defined at task-attempt granularity: a task kind determines the reusable model structure, a task instance supplies intrinsic work features, and a task attempt is the concrete execution in which demand, occupancy, observations, outcomes, and residuals are realized.

Let $a \in \mathcal{A}$ be a task attempt. The attempt executes task instance $i(a) \in \mathcal{I}$, whose task kind is $k(i(a)) \in \mathcal{K}$. For readability, we write

$$k_a = k(i(a))$$

for the task kind of attempt a. We also write $\mathbf{x}_a$ for the intrinsic feature vector of the task instance executed by a, and $\mathbf{c}_a$ for the execution-context information available for that attempt. The vector $\mathbf{x}_a$ may include properties such as payload size, record count, object count, partition size, batch size, operation mode, or domain metadata. The vector $\mathbf{c}_a$ may include worker state, node pressure, dependency health, network condition, disk pressure, cache state, retry state, tenant pressure, or domain-specific state.

The distinction between k_a, $\mathbf{x}_a$, and $\mathbf{c}_a$ is central. A task kind may define a reusable cost structure, but it does not determine one universal cost value. Two attempts of the same task kind may execute the same logical operation with similar intrinsic features and still differ because they observe different execution contexts. Conversely, two attempts may execute under similar context but differ because their intrinsic features represent different amounts of work. The cost estimate for an attempt is therefore conditional on the task kind, the intrinsic features of the task instance, and the execution context of the attempt.

Formally, the cost model estimates two attempt-indexed vectors. The first vector contains active resource-demand estimates:

$$\widehat{\mathbf{D}}(a) = \left\{ \widehat{D}_r(a) \mid r \in \mathcal{R} \right\},$$

where $\mathcal{R}$ is the set of resource-demand dimensions. The second vector contains worker-occupancy estimates:

$$\widehat{\mathbf{U}}(a) = \left\{ \widehat{U}_o(a) \mid o \in \mathcal{O} \right\},$$

where $\mathcal{O}$ is the set of worker-occupancy dimensions. The attempt-level cost estimate is the ordered pair

$$\widehat{\mathbf{C}}(a) = \left(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a) \right).$$

This notation intentionally treats cost as a structured object rather than as a single scalar.

The notation separates model structure from model evaluation. The task kind k_a determines which resource and occupancy dimensions are relevant and which intrinsic or contextual features should be considered. The attempt a provides the concrete values used to evaluate that structure under a particular context. This separation allows later sections to associate Task Behavior Graphs with task kinds or task families while still producing attempt-level estimates for concrete executions.

A scalar value may be derived from $\widehat{\mathbf{C}}(a)$ by a downstream policy, but such a value is not the cost model itself. For example, an admission policy, ranking policy, alerting rule, or profiling-policy selector may apply a projection

$$s_\rho(a) = \pi_\rho\left(\widehat{\mathbf{D}}(a), \widehat{\mathbf{U}}(a) \right)$$

under some policy ρ. The projection $s_\rho(a)$ is policy-specific: it depends on the decision being made, the available capacity, and the relative importance assigned to different dimensions. The

cost model therefore preserves the vector-valued estimate and does not collapse demand and occupancy into a universal scalar.

Operational risk is also not part of the cost object. Risk may use demand and occupancy estimates, but it captures the consequence of underestimating, misclassifying, or mishandling an attempt. In this paper, cost estimates are inputs to later uncertainty, risk, and profiling-policy analysis; they are not the risk model itself.

5.2 Resource Demand and Worker Occupancy

The attempt-level cost object contains two distinct families of quantities: active resource demand and worker occupancy. Both are attempt-level cost dimensions, but they describe different operational effects. Resource demand describes active work required or consumed by the attempt. Worker occupancy describes bounded execution capacity held by the attempt over time. The two families may be correlated during execution, but they are not interchangeable and should not be collapsed into a single load value.

For a resource dimension $r \in \mathcal{R}$, $\widehat{D}_r(a)$ denotes the estimated active resource demand of attempt a. Resource-demand dimensions describe resources that the attempt actively requires or consumes, such as CPU time, memory peak, allocation volume, disk reads, disk writes, network input, network output, or accelerator usage. These dimensions are attempt-attributed: they describe the work associated with the attempt itself rather than the ambient state of the worker, node, dependency, or queue.

For an occupancy dimension $o \in \mathcal{O}$, $\widehat{U}_o(a)$ denotes the estimated bounded-capacity occupancy of attempt a. Occupancy dimensions describe scarce execution capacity held by the attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant-level concurrency time. These quantities are also attempt-attributed, but they measure capacity held rather than active resource work. A dependency-bound attempt may consume little CPU while still holding a worker slot or dependency connection for a long duration.

This distinction separates three concepts that are often conflated in queue systems: demand, occupancy, and pressure. Demand and occupancy are properties estimated or observed for the attempt. Pressure is a property of the execution context. For example, CPU time is an active resource-demand dimension of an attempt, whereas CPU saturation on the node is an execution-context factor. Similarly, disk write bytes are a resource-demand dimension, whereas disk queue depth or storage saturation are context factors. Worker-pool pressure describes the state of the pool, such as the fraction of busy worker slots; worker-slot time describes the capacity held by a particular attempt. The former may influence the latter, but it is not the same quantity.

The same separation applies to waiting and elapsed time. Dependency health, network condition, connection-pool pressure, and retry state are context factors. Dependency wait time is an intermediate or observed waiting signal. Wall-clock duration is an elapsed-time outcome. Worker-slot time and connection hold time are occupancy dimensions only when the corresponding bounded capacities remain held during that elapsed interval. Therefore, a long dependency wait does not by itself imply equal worker occupancy. If a worker blocks while waiting, the wait may increase worker-slot time. If the worker releases its slot and resumes later, the end-to-end wall-clock duration may be large while worker-slot occupancy remains small.

Two examples illustrate why both families are needed. A compute-dominant transformation may have high CPU time and allocation volume but release its worker slot quickly if the computation is short and does not block on external dependencies. Its active resource demand is high, while its occupancy may remain moderate. Conversely, an external-service or replication attempt may have low CPU demand but high worker-slot time, connection hold time, or lease time when a dependency is slow, a peer is degraded, or retries introduce backoff. In that case, active resource demand alone would understate the cost imposed on the worker pool.

The model therefore keeps resource demand and worker occupancy as separate vectors. A downstream policy may later combine selected dimensions into a scalar score, but such a score is a policy-specific projection. The cost model itself preserves the distinction because different dimensions have different units, different causes, different aggregation rules, and different operational consequences.

This separation prepares the decomposition introduced next. Intrinsic task features may define a baseline for some resource and occupancy dimensions, while execution context may amplify or suppress them. The same context factor can affect different dimensions in different ways: degraded network conditions may have little effect on a CPU-bound transform, but may strongly increase worker-slot time for a replication or external-service attempt. For this reason, baseline demand, contextual amplification, and additive overhead must be modeled per dimension rather than through one global cost scalar.

5.3 Baseline, Amplification, and Additive Overhead

The preceding subsection separated the cost object into resource-demand and worker-occupancy dimensions. This subsection describes how each dimension is estimated. The model decomposes an attempt-level cost estimate into three parts: intrinsic baseline, contextual amplification, and additive overhead. The baseline represents work implied by the task kind and intrinsic features. The amplification term represents the effect of execution context on that baseline. The overhead term represents costs that are better modeled as additional work or additional held capacity rather than as a multiplier of the baseline.

Let

$$\mathcal{Q} = \mathcal{R} \cup \mathcal{O}$$

be the set of cost dimensions. For $q \in \mathcal{Q}$, let $\widehat{Y}_q(a)$ denote the estimated value for dimension q during attempt a. If $q \in \mathcal{R}$, then $\widehat{Y}_q(a)$ corresponds to a component of $\widehat{\mathbf{D}}(a)$. If $q \in \mathcal{O}$, then $\widehat{Y}_q(a)$ corresponds to a component of $\widehat{\mathbf{U}}(a)$. The dimension-generic estimate is

$$\widehat{Y}_q(a) = \widehat{B}_q(k_a, \mathbf{x}_a) \cdot \widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a) + \widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a), \qquad q \in \mathcal{Q}.$$

Here $\widehat{B}_q$ is the estimated intrinsic baseline, $\widehat{A}_q$ is the estimated contextual amplification factor, and $\widehat{O}_q$ is the estimated additive overhead for dimension q.

The baseline term $\widehat{B}_q(k_a, \mathbf{x}_a)$ captures the part of cost that can be attributed to the task kind and intrinsic work features. For a compute transformation, record count and algorithm mode may determine a baseline for CPU time and allocation volume. For a replication task, payload size, changed-object count, or partition size may determine a baseline for disk reads, network output, and commit work. The baseline is not a constant attached to the task kind alone. It is task-kind-specific, but it is evaluated using the intrinsic feature values of the task instance executed by the attempt.

The amplification term $\widehat{A}_q(k_a, \mathbf{x}_a, \mathbf{c}_a)$ captures how execution context modifies the baseline for dimension q. The term may be greater than one, close to one, or, in some cases, less than one. A degraded peer, unstable network, saturated dependency, high disk pressure, cold cache, or high retry state may amplify some dimensions. A warm cache, healthy peer, or local data placement may suppress others. The amplification term depends on both intrinsic features and context because their interaction matters. Poor network conditions may have little effect on a small transfer but dominate a large replication attempt.

The overhead term $\widehat{O}_q(k_a, \mathbf{x}_a, \mathbf{c}_a)$ captures cost that should not be treated as a proportional multiplier of the baseline. Examples include fixed setup, deserialization, connection establishment, lease-extension bookkeeping, acknowledgement, commit, cleanup, and accounting or profiling overhead. Some overheads are nearly constant for a task kind; others vary with context or intrinsic features. Modeling them additively prevents the model from forcing every effect into the baseline-amplification product.

This decomposition is an engineering model, not an independence claim. It does not assert that intrinsic work and execution context are statistically independent, nor does it require that all dimensions behave linearly. Its role is to keep three different mechanisms separate: work implied by the task, contextual effects that modify that work, and additional overhead that is not well represented as a multiplier. Later sections use influence domains and Task Behavior Graphs to make these mechanisms explicit for each task kind.

The decomposition is dimension-specific. A single context factor may amplify one dimension, suppress another, and leave a third unchanged. For example, network instability may strongly increase worker-slot time for a replication attempt, moderately increase network retransmission volume, and have almost no direct impact on CPU demand for a CPU-bound transform. Consequently, the model should not use one global amplification factor for the whole attempt. It should estimate or declare amplification per relevant dimension.

5.4 Residuals and Observed Usage

The cost model distinguishes estimated values from observed values. Before or during execution, the profiler produces an estimate $\widehat{Y}_q(a)$ for a cost dimension q. After execution, the system may observe an actual value $Y_q(a)$, depending on instrumentation coverage and measurement quality. The residual for dimension q is

$$\epsilon_q(a) = Y_q(a) - \widehat{Y}_q(a).$$

For $q \in \mathcal{R}$ this residual corresponds to unexplained resource usage or demand behavior. For $q \in \mathcal{O}$ it corresponds to unexplained worker-occupancy behavior.

Residuals are not merely accounting errors. They are signals about the relation between the model and the workload. A large residual may indicate missing intrinsic features, stale execution-context observations, hidden dependency state, an incorrect task-kind classification, model misspecification, measurement error, or a rare tail event. A small residual does not prove causal correctness; it only indicates that the estimate was close to the observed value for that attempt and dimension.

Observed values also require semantic care. A raw measurement may not directly correspond to a cost dimension. For example, wall-clock duration may include CPU execution, dependency wait, local scheduling delay, retry backoff, and commit time. It becomes an occupancy measurement only for the capacity that remains held during the corresponding interval. Similarly, node-level CPU utilization may be useful context, but it is not by itself the observed CPU usage of a particular attempt. The model therefore treats observed usage and observed occupancy as attempt-attributed values, not as arbitrary raw telemetry streams.

Residuals connect the cost model to later uncertainty analysis. If residuals are large, systematic, or concentrated in particular contexts, the model may need additional features, a revised graph, a different task taxonomy, or a higher profiling level. If residuals grow after a deployment or dependency change, they may indicate drift. This section only defines residuals as differences between observed and estimated cost dimensions; Section 12 uses them as one input to confidence, uncertainty, and risk analysis.

The residual definition also supports cold-start and partial-observability cases. For a new task kind, the system may begin with coarse estimates and wide uncertainty. As observations accumulate, residuals reveal which dimensions are poorly explained. For a partially instrumented task, residuals may be available only for some dimensions. The absence of a residual for a dimension should not be interpreted as model accuracy; it may simply mean that the dimension was not observed.

5.5 Distributional Cost Estimates

Attempt-level cost is often variable even within the same task kind and feature bucket. A single mean estimate can hide operationally important behavior, especially for tasks with retries,

dependency waits, lock contention, cache misses, or degraded peers. The cost model therefore allows each dimension to be estimated as a distribution or as a set of summary statistics rather than only as a point value.

For a cost dimension $q \in \mathcal{Q}$ and a quantile level $p \in (0, 1)$, we write

$$\widehat{Y}_{q,p}(a)$$

for a quantile estimate of dimension q for attempt a. For example, $\widehat{Y}_{q,0.50}(a)$ may represent a median estimate and $\widehat{Y}_{q,0.99}(a)$ may represent a high-tail estimate. The paper does not require one particular estimator. A system may use empirical quantiles, histograms, sketches, parametric distributions, conservative envelopes, or model-specific prediction intervals.

Distributional estimates are useful because different decisions depend on different parts of the distribution. A mean CPU-time estimate may be sufficient for coarse accounting of stable compute tasks. A high-percentile worker-slot estimate may be more relevant for capacity protection when rare attempts can hold workers for long periods. A high-percentile connection-hold estimate may be more relevant for dependency protection than average network bytes. The cost model therefore preserves the dimension and distributional form before any policy-specific projection is applied.

Distributional estimates also prevent a common modeling error: treating variability as if it were only noise around a single expected value. In queue-worker systems, variability may reflect distinct modes. A task may be fast when cache is warm and slow when cache is cold; a replication attempt may be short when peers are healthy and long when retries occur; an external-service attempt may be bimodal because dependency timeouts create a second latency mode. The model should allow such behavior to remain visible rather than compress it into one average.

This section does not define entropy, confidence, or risk metrics. It only states that cost estimates may be distributional. Later sections may derive uncertainty indicators from residuals, tail ratios, drift, missing features, multimodality, or insufficient sample size. This separation keeps the cost model focused on what is estimated, while later sections describe how reliable or risky those estimates are.

5.6 Boundaries of the Cost Model

The cost model defines the demand and occupancy quantities that the profiling model estimates. It does not define the full behavior model. In particular, it does not define the complete set of influence domains, the taxonomy of task behavior classes, the structure of Task Behavior Graphs, the raw-observation to feature-mapping pipeline, or the profiling-policy selection algorithm. Those sections build on the cost model but have different responsibilities.

The first boundary is between cost and execution context. Execution context contains input-side conditions such as node pressure, worker-pool pressure, dependency health, network condition, cache state, retry state, and domain state. These factors may influence the cost estimate, but they are not cost outputs themselves. For example, disk pressure may amplify disk-write duration, but disk pressure is not disk-write demand. Worker-pool pressure may affect waiting and admission decisions, but it is not worker-slot time for a particular attempt.

The second boundary is between cost and observed symptoms. Wall-clock duration, latency, timeout count, retry count, and queue delay may provide evidence about cost, but they are not automatically cost dimensions. Their role depends on what they measure and which capacity remains held. A long wall-clock duration may reflect high CPU time, dependency wait, local scheduling delay, retry backoff, or released-slot asynchronous waiting. The model should assign such observations to explicit semantic roles before using them as cost evidence.

The third boundary is between cost and risk. Cost describes estimated demand, occupancy, and overhead for an attempt. Risk describes the consequence of getting that estimate wrong

or acting on it incorrectly. Long worker-slot time may contribute to queue starvation risk, and high network demand may contribute to dependency overload risk, but the risk depends on system state, policy, SLOs, tenant importance, and blast radius. Risk therefore consumes cost estimates; it is not part of the cost object.

The fourth boundary is between cost and profiling policy. Profiling overhead can be represented as an additive overhead dimension when instrumentation is enabled, but the decision to enable that instrumentation belongs to the profiling-policy layer. A task may be expensive enough, uncertain enough, or consequential enough to justify more detailed observation. The cost model supplies the dimensions that such a policy may use; it does not decide the policy by itself.

These boundaries define the role of this section in the paper. The cost model specifies what is estimated at attempt granularity. Section 6 assigns semantic roles to the inputs, outputs, symptoms, and risks that participate in those estimates. Section 8 connects those roles into task-kind-specific graphs. Section 12 uses residuals and distributional estimates to reason about uncertainty and risk. Section 13 uses those estimates to select observation strategies under an overhead budget.

References

[1] Yue Cheng, Ali Anwar, and Xuejing Duan. Analyzing alibaba's co-located datacenter workloads. In *2018 IEEE International Conference on Big Data*, Big Data '18. IEEE, 2018.

[2] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[3] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, OSDI '04, pages 137–150. USENIX Association, 2004.

[4] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13. ACM, 2013.

[5] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14. ACM, 2014.

[6] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '11. USENIX Association, 2011.

[7] OpenTelemetry. Opentelemetry documentation: Sampling. `https://opentelemetry.io/docs/concepts/sampling/`, 2025. Accessed 2026-05-03.

[8] OpenTelemetry. Opentelemetry documentation: Performance. `https://opentelemetry.io/docs/zero-code/java/agent/performance/`, 2026. Accessed 2026-05-03.

[9] OpenTelemetry. Opentelemetry specification: Metrics data model. `https://opentelemetry.io/docs/specs/otel/metrics/data-model/`, 2026. Accessed 2026-05-03.

[10] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing*, SoCC '12. ACM, 2012.

[11] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.

[12] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15. ACM, 2015.

[13] Qi Zhang, Joseph L. Hellerstein, and Raouf Boutaba. Characterizing task usage shapes in google compute clusters. In *Proceedings of the 5th International Workshop on Large Scale Distributed Systems and Middleware*, LADIS '11. ACM, 2011.

A Extended Terminology

This appendix provides the complete terminology reference used by the paper. Section 2 intentionally keeps only the core terms needed for the main model. The extended glossary is organized by semantic area so that the main text can introduce concepts progressively while still keeping a precise reference for task entities, behavior classes, execution context, observations, influence domains, resource demand, worker occupancy, uncertainty, risk, graph structure, profiling policy, manifests, and queue-worker lifecycle stages.

A.1 Task Entities

Table 3: Task entity terms.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.

Term	Definition
task kind	The semantic type of work submitted to a queue, such as copying a replication partition, rebuilding a cache entry, resizing an image, repairing a database record, dispatching a webhook, or validating a document. A task kind describes what work is requested, but it does not fully determine how that work behaves under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution attempt of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource demand, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
task family	A group of task kinds that share a similar behavior-graph structure or profiling model. A task family can be used when one graph template applies to several related task kinds.
task identity	The identifying metadata of a task, including task kind, task version, queue, producer, tenant, priority, and deadline. Task identity identifies the work but does not itself define the cost of executing it.
task version	The version of a task kind, handler contract, payload schema, or behavior manifest. Task versions are useful for compatibility between historical profiles, manifests, and schema versions.
queue	The queue through which a task instance is submitted, stored, reserved, and eventually consumed by a worker. A queue name may provide useful context, but it is not a complete behavior model.
producer	The component that creates and submits a task instance to a queue. The producer may provide metadata or declared hints, but it does not fully determine the execution context.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
worker pool	A set of workers that execute task attempts. Worker-pool capacity and worker-pool occupancy are central to queue-worker profiling.

Term	Definition
tenant	A user, customer, namespace, application, or logical group that consumes execution capacity. Tenants are relevant for isolation, fairness, consequence analysis, and operational risk.
tenant class	A category of tenants based on SLO, priority, plan, criticality, or operational risk. A tenant class is distinct from a task kind.
priority	A scheduling or ordering attribute assigned to a task instance. Priority may influence profiling or admission decisions, but it is not itself resource demand or operational risk.
deadline	A completion target or time constraint associated with a task instance. A deadline may affect consequence and risk, but it is distinct from observed latency.

A.2 Task Behavior

Table 4: Task behavior terms.

Term	Definition
task behavior	The observed or estimated execution pattern of a task kind, task instance, or task attempt under a particular execution context. Task behavior includes resource demand, worker occupancy, retry behavior, dependency wait, outcome, uncertainty, and risk-relevant effects.
behavior profile	A summarized description of how a task kind behaves under a class of execution contexts. A behavior profile may include demand distributions, occupancy distributions, dominant behavior classes, uncertainty, risk class, and relevant features.
behavior class	An analytical category derived from observed or estimated behavior. Examples include compute-dominant, dependency-bound, retry-amplifying, worker-slot-heavy, payload-sensitive, state-dependent, or high-tail-risk behavior.
behavior mode	A context-dependent mode of behavior for a task kind. The term emphasizes that the same task kind may behave differently under different execution contexts.
compute-dominant behavior	Behavior in which CPU demand is the dominant active resource dimension. A task kind may exhibit compute-dominant behavior without being intrinsically a CPU task in all contexts.

Term	Definition
memory-pressure-sensitive behavior	Behavior that is sensitive to memory pressure, allocation rate, working-set size, or garbage-collection effects. This behavior is not necessarily identical to being memory-bound.
disk-IO-dominant behavior	Behavior in which disk reads, disk writes, storage throughput, or I/O wait dominates execution. Disk I/O demand is distinct from disk pressure in the execution environment.
network-sensitive behavior	Behavior that changes substantially with network condition, such as packet loss, jitter, retransmissions, bandwidth changes, or timeouts.
dependency-bound behavior	Behavior constrained primarily by external dependencies, such as databases, APIs, remote services, rate limits, quotas, or connection pools.
retry-amplifying behavior	Behavior in which retries substantially increase work, worker occupancy, or operational risk. Retry count may be a symptom or outcome rather than an independent cause.
worker-slot-heavy behavior	Behavior with high worker-slot time relative to active resource usage. This class is important for queue workers because a task may consume little CPU but hold scarce worker capacity for a long time.
payload-sensitive behavior	Behavior that depends strongly on payload size, input shape, record count, object count, or batch size. Payload-sensitive behavior can still be uncertain if context or hidden state varies.
state-dependent behavior	Behavior that depends on domain-specific or internal system state, such as replica divergence, cache state, compaction debt, segment fragmentation, or dirty data volume.
high-tail-risk behavior	Behavior in which rare slow executions or high-percentile outcomes are operationally important. High-tail-risk behavior is distinct from high average cost.
stable behavior	Behavior with low residual error, low drift, and low variability under comparable execution contexts. Stable behavior may still be expensive.
unstable behavior	Behavior with high variability, drift, residual error, multimodality, or context sensitivity. Unstable behavior is not necessarily high-cost in every attempt.

A.3 Execution Context

Table 5: Execution-context terms.

Term	Definition
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.
context factor	A feature of the execution context that may influence task behavior. Context factors are modeled as influences, not automatically as proven causes.
worker state	The local state of the worker during a task attempt, including local runtime pressure, running work, memory state, garbage-collection pressure, and local scheduling delay.
worker pressure	Pressure on a worker or worker pool, such as busy worker slots, saturated local queues, high utilization, or local scheduling delay.
node pressure	Pressure on the host, virtual machine, container, or node on which the worker runs. Examples include CPU saturation, memory pressure, disk pressure, and network saturation.
dependency health	The operational condition of an external dependency, such as latency, timeout rate, error rate, saturation, quota pressure, or availability.
dependency capacity	The available capacity of an external dependency, such as connection-pool capacity, rate-limit budget, service quota, or downstream throughput.
network condition	The state of the network path relevant to a task attempt, including round-trip time, jitter, packet loss, retransmissions, bandwidth, timeouts, and availability.
network instability	A normalized feature representing unstable network behavior. It may be derived from packet loss, jitter, retransmissions, timeout rate, bandwidth variation, or recent network failures.
replica state	The state of a replica, shard, peer, or replica set involved in a task attempt. Replica state is relevant to replication, repair, synchronization, and consistency tasks.
replica health	A normalized feature representing the operational condition of a replica or peer. Replica health is related to, but distinct from, replication lag.
replication lag	An observation or signal describing how far a replica is behind another source of truth. It may be measured in bytes, records, operations, versions, or time.

Term	Definition
disk pressure	Pressure on the storage subsystem, such as queue depth, read latency, write latency, I/O wait, or saturation. Disk pressure is an execution-context factor, not the same as disk I/O demand.
cache state	The condition of a cache relevant to a task attempt, such as warm, cold, stale, evicted, partially populated, or inconsistent.
cache warmth	A normalized feature representing how prepared or populated a cache is for the task attempt. Cache warmth can influence resource demand and latency.
data locality	The proximity of required data to the worker or execution environment. Examples include local, remote, cross-zone, cross-region, or external storage access.
tenant pressure	Pressure or quota usage associated with a tenant, such as used concurrency, rate-limit consumption, or tenant-level backlog.
retry state	The state of the retry cycle for a task instance or attempt, including attempt number, previous errors, backoff state, retry budget, and retry history.
hidden system dynamics	Unobserved or partially observed system state that affects task behavior, such as internal dependency queues, lock contention, cache eviction history, data fragmentation, noisy neighbors, kernel behavior, or runtime effects. Hidden dynamics are represented through residual error, uncertainty, and reduced confidence when not captured by known features.

A.4 Observations, Signals, and Features

Table 6: Observation, signal, and feature terms.

Term	Definition
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.
raw observation	An unnormalized measurement or event before interpretation. A raw observation does not by itself have stable model meaning.
metric	A time-series, counter, gauge, or histogram observation. Metrics are one kind of observation; traces, logs, profiles, and domain events are observations too.

Term	Definition
counter	A metric that counts events, operations, bytes, retries, failures, or other monotonic quantities. A counter may represent a symptom or outcome rather than a root influence.
gauge	A metric representing a current value, such as queue depth, active workers, available connections, or current memory usage.
histogram	A metric representing a distribution of observed values, such as durations, latencies, sizes, or per-attempt costs.
trace	An execution trace composed of spans. A trace exposes execution structure and timing, but it does not by itself define task-kind-specific relevance, causality, feature mapping, or profiling policy.
span	A timed unit inside a trace representing an operation, processing stage, dependency call, or instrumented region. A span is not the same as a task attempt.
log	A structured or unstructured event record emitted by a system. Logs may expose status, error classes, domain events, or diagnostic information.
runtime profile	A CPU, heap, allocation, lock, blocking, or runtime-level profile. A runtime profile is distinct from a behavior profile.
signal	An interpreted, aggregated, or derived observation that indicates a condition relevant to the model. A signal is an intermediate layer between raw observation and canonical feature.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
semantic feature	An informal synonym for a feature with explicit semantic meaning. This paper uses the term canonical feature when referring to model inputs with defined type, range, unit, influence domain, and intended meaning.
feature mapping	The process of converting raw observations, signals, task metadata, and context into canonical features. Feature mapping is separate from graph evaluation.
normalization	The process of converting values into a defined type, unit, scale, bucket, or range. For example, network measurements may be normalized into an instability score in $[0, 1]$.
aggregation	The process of combining observations over a time window, group, task kind, attempt set, or distribution. Examples include sum, average, maximum, p95, and p99.

Term	Definition
feature registry	A registry of canonical features, including names, types, ranges, units, influence domains, and intended meaning.
metric descriptor	A descriptor of a raw metric, including source, unit, label set, aggregation, cardinality, and mapping to canonical features.
feature descriptor	A descriptor of a canonical feature, including type, range, unit, semantic meaning, influence domain, and expected use in the model.
cardinality	The number of distinct time series or label combinations produced by a metric. Cardinality affects telemetry overhead and storage cost.
high-cardinality label	A metric label that can create a large or unbounded number of series, such as task id, user id, object id, URL, or unbounded tenant label.
observation window	The time window over which observations are aggregated or interpreted. For example, network instability may be computed over the last 30 seconds.
freshness	How recent and applicable an observation is to the current task attempt. Stale observations may reduce confidence.

A.5 Influence Domains

Table 7: Influence domain terms.

Term	Definition
influence domain	A semantic role assigned to a feature, observation, output, or model concept in the task behavior model. Influence domains are not metric domains: they group model elements by role rather than by telemetry source.
task identity domain	The influence domain containing identifiers and metadata that describe which task is being executed, such as task kind, version, queue, tenant, producer, priority, and deadline.
intrinsic demand domain	The influence domain describing work contained in the task itself, such as payload size, record count, object count, partition size, input format, operation mode, or batch size.
execution context domain	The influence domain describing the environment and state under which the task attempt executes, such as network condition, dependency health, replica state, disk pressure, cache warmth, worker pressure, and retry state.

Term	Definition
resource demand domain	The influence domain describing active resource dimensions required or consumed by the task attempt, such as CPU time, memory peak, disk I/O, network I/O, runtime allocations, or accelerator usage. Waiting on dependencies is not active resource demand by itself, although it can contribute to worker occupancy and operational risk.
worker occupancy domain	The influence domain describing bounded execution capacity held by the task attempt, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant concurrency time. Wait-related dimensions such as dependency wait time may contribute to this domain when a task keeps bounded worker capacity while waiting.
outcome domain	The influence domain describing how the task attempt completed, such as success, failure, timeout, cancellation, retryable error, non-retryable error, dead-lettering, or lease expiration.
uncertainty domain	The influence domain describing unreliability or unpredictability in the model estimate or behavior profile, such as residual error, drift, variance, tail ratio, multimodality, missing features, or low sample count.
risk domain	The influence domain describing the consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Examples include queue starvation, retry storm, SLO violation, replication lag, tenant impact, and cascading overload.
profiling observation domain	The influence domain describing what is observed, where it is observed, at what detail level, and under which overhead budget.

A.6 Resource Demand, Occupancy, Cost, and Overhead

Table 8: Resource demand, occupancy, cost, and overhead terms.

Term	Definition
resource demand	The expected or required active resource need of a task attempt along a resource dimension. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.

Term	Definition
active resource usage	Actual use of active resources such as CPU, memory, disk, network, runtime allocation, or accelerator resources. Active resource usage is distinct from waiting and bounded-capacity occupancy.
CPU time	The amount of CPU execution time consumed by a task attempt. CPU time is not the same as wall-clock duration.
CPU pressure	Pressure on CPU capacity in the worker, container, node, or co-located environment. CPU pressure is an execution-context factor, not the same as task CPU demand.
memory peak	The maximum memory used by a task attempt during execution. Memory peak is a resource-usage or resource-demand dimension.
memory pressure	Pressure on available memory in the worker, container, node, or runtime environment. Memory pressure is an execution-context factor.
allocation behavior	The allocation volume, allocation rate, allocation pattern, or allocation-induced runtime effect of a task attempt. Allocation behavior may influence memory pressure, garbage collection, and CPU time.
disk I/O	Disk reads, writes, operations, bytes, or I/O wait associated with a task attempt. Disk I/O demand is distinct from disk pressure.
network I/O	Network bytes, packets, requests, or transfer volume associated with a task attempt. Network I/O is distinct from network instability.
wait time	Time spent waiting rather than actively consuming a hardware or runtime resource. Wait time may be caused by dependencies, locks, network transfer, backoff, throttling, local scheduling delay, or other blocking conditions.
dependency wait time	Time spent waiting for an external dependency, such as a database, API, remote service, or storage system. Dependency wait time is not active resource demand by itself, but it can increase wall-clock duration and worker occupancy when bounded worker capacity remains held during the wait.
worker occupancy	Bounded execution capacity held by a task attempt over time. Worker occupancy is a first-class cost component in queue-worker systems.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension.
lease time	The time during which a task lease or reservation is held. Lease time may differ from worker-slot time.

Term	Definition
connection hold time	The time during which a task attempt holds a connection to a dependency, database, broker, or external service.
lock hold time	The time during which a task attempt holds a lock. Lock hold time can affect other tasks and may contribute to contention.
lock wait time	The time spent waiting to acquire a lock. Lock wait time is distinct from lock hold time.
semaphore hold time	The time during which a task attempt holds a semaphore, concurrency token, or bounded execution permit.
tenant concurrency time	The time during which a task attempt consumes a tenant-level concurrency slot. This dimension is relevant to fairness and isolation.
wall-clock duration	Elapsed time from attempt start to attempt finish. Wall-clock duration is often related to worker-slot time, but the two are not necessarily identical.
latency	Delay observed by a caller, queue, worker, dependency, or system boundary. Latency is an outcome or symptom, not resource demand by itself.
service time	The time a task attempt spends being served or executed, excluding queue wait time. The exact boundary must be defined for the queue-worker system being analyzed.
queue wait time	The time between enqueue and dequeue or execution start. Queue wait time is distinct from worker-slot time.
task cost	An umbrella term for resource demand, worker occupancy, and execution overhead associated with a task attempt. Operational risk is not part of task cost.
execution overhead	Overhead inherent to executing a task attempt, such as serialization, input fetching, setup, acknowledgement, commit, cleanup, or retry bookkeeping.
profiling overhead	Overhead caused by profiling and instrumentation, including tracing, metrics, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export.
telemetry overhead	CPU, memory, allocation, network, storage, or cardinality cost introduced by telemetry collection and export.

A.7 Uncertainty, Confidence, Residuals, and Risk

Table 9: Uncertainty, confidence, residual, and risk terms.

Term	Definition
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
entropy	A possible measure or family of measures for unpredictability. The term should only be used after specifying what distribution, signal, or behavior process is being measured.
confidence	The degree of trust in an estimate, behavior profile, graph edge, feature mapping, or model assumption. Confidence is distinct from uncertainty.
residual error	The part of observed behavior not explained by the current model estimate. Residual error exposes missing features, hidden influences, drift, or model mismatch.
prediction error	The difference between predicted and observed value for a resource, occupancy, duration, outcome, or risk-relevant dimension.
drift	A change in task behavior, feature mapping, graph relationship, context, or profile over time. Drift is a source of uncertainty and reduced confidence.
variance	The statistical spread of observed values. Variance is a variability signal but is not identical to entropy.
tail ratio	A ratio comparing a tail quantile with a central quantile, such as p95/p50 or p99/p50. Tail ratios help identify high-tail-risk behavior.
multimodality	The presence of multiple behavioral regimes or modes in the observed behavior of a task kind or task family.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is separate from task cost.
consequence	The severity of the effect caused by an incorrect estimate or control decision, such as delayed processing, queue starvation, SLO violation, replication lag, or cascading overload.
blast radius	The scope affected by an error or overload event, such as a single task, queue, worker pool, tenant, replica set, dependency, cluster, or control plane.
SLO violation risk	The operational risk that a task attempt, task kind, queue, tenant, or service will violate a service-level objective.
queue starvation risk	The risk that long-running or high-occupancy tasks delay or prevent other tasks from receiving execution capacity.

Term	Definition
retry amplification risk	The risk that retries increase work, dependency pressure, queue occupancy, or worker occupancy enough to worsen overload.
cascade overload risk	The risk that local overload propagates to other queues, workers, dependencies, tenants, or control loops.

A.8 Task Behavior Graphs

Table 10: Task Behavior Graph terms.

Term	Definition
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph is not assumed to prove causality by itself; it makes modeled influence assumptions explicit.
graph node	An element of a Task Behavior Graph representing a canonical feature, output, dimension, intermediate signal, or model concept. Each node belongs to one or more influence domains.
graph edge	A modeled influence relationship between two graph nodes. A graph edge is not a proven causal link unless validated by additional evidence.
influence role	The semantic role of a node or edge in the graph, such as baseline demand, contextual amplification, additive overhead, symptom, outcome link, risk propagation, uncertainty indicator, or profiling trigger.
baseline demand edge	An edge that connects an intrinsic feature to a baseline resource-demand or occupancy estimate. For example, payload size may contribute to network I/O or disk I/O baseline demand.
contextual amplification edge	An edge by which an execution-context factor amplifies or suppresses demand, occupancy, uncertainty, or risk.
additive overhead edge	An edge that represents additional fixed or variable overhead, such as setup, serialization, acknowledgement, commit, or retry bookkeeping.
symptom edge	An edge that connects an influence to a symptom or observed effect, such as network degradation contributing to retries or longer wall-clock duration.
outcome link	An edge that connects modeled behavior to an outcome, such as timeout, failure, success, cancellation, or dead-lettering.

Term	Definition
risk propagation edge	An edge that connects demand, occupancy, uncertainty, or outcome to an operational-risk dimension.
uncertainty indicator	A node or edge role indicating that a signal changes uncertainty or confidence rather than directly changing resource demand.
sensitivity	The estimated strength of a relationship between graph nodes. Sensitivity may be expert-defined, learned, calibrated, or represented as a qualitative level.
relevance	Whether a feature or relationship matters for a task kind, task family, or behavior profile. Relevance is distinct from sensitivity.
graph evaluation	The process of applying a Task Behavior Graph to canonical features and execution context in order to estimate demand, occupancy, uncertainty, residuals, and risk.
graph template	A reusable graph structure that can be specialized for multiple related task kinds within a task family.
model assumption	An explicit assumption encoded in the graph, feature mapping, or policy. Model assumptions should be validated, calibrated, or revised when observations contradict them.

A.9 Profiling and Observation Policy

Table 11: Profiling and observation-policy terms.

Term	Definition
profiling	The process of collecting, interpreting, and using observations to estimate task behavior. In this paper, profiling is broader than CPU profiling.
task profiling	Profiling logical task attempts in queue-worker systems in order to estimate resource demand, worker occupancy, uncertainty, and operational risk.
observation strategy	A decision about what to observe, where to observe it, how often to observe it, and under what overhead budget.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context.
accounting	Recording basic execution facts, such as start time, finish time, status, duration, input size, output size, resource usage, or occupancy.
minimal accounting	A low-overhead accounting level that records only essential task facts, such as start time, finish time, status, and basic size information.

Term	Definition
detailed accounting	An accounting level that records more detailed resource, occupancy, stage, dependency, and outcome information.
tracing	Capturing execution structure and timing through traces and spans. Tracing is an observation method, not a behavior model by itself.
sampled tracing	Tracing only a subset of tasks, attempts, or traces in order to control overhead.
tail-aware profiling	A profiling strategy that preferentially observes slow, failed, high-risk, or tail-heavy attempts.
diagnostic profiling	Deep and usually expensive profiling used to investigate unexplained behavior, high residual error, drift, or model failure.
isolated execution profiling	Profiling in a more controlled or isolated execution environment in order to reduce interference or better attribute behavior.
cooperative instrumentation	Instrumentation in which worker or task code explicitly emits structured observations such as spans, timers, byte counts, record counts, dependency calls, stage boundaries, and domain-specific signals.
sampling	Selecting a subset of tasks, attempts, events, spans, traces, or profile samples for observation or retention.
head sampling	A trace-sampling strategy that decides whether to sample before the full trace is observed.
tail sampling	A trace-sampling strategy that decides whether to sample after seeing all or most of a trace.
observation point	A point in the task lifecycle where observations can be collected, such as enqueue, dequeue, reserve, start, dependency call, retry, commit, acknowledge, or finish.
profiling budget	The allowed overhead for profiling and telemetry, including CPU, memory, allocation, storage, network, and cardinality costs.
hot path	The latency- or throughput-sensitive path of task execution where added instrumentation overhead can affect production behavior.

A.10 Manifests, Registries, and Domain Packs

Table 12: Manifest, registry, and domain-pack terms.

Term	Definition
Task Behavior Manifest	A declarative description of the behavior model for a task kind or task family. It may describe features, domains, graph nodes, graph edges, observation requirements, profiling policy defaults, known gaps, and versioning.
manifest	A machine-readable or human-readable specification used to declare part of the task behavior model.
domain pack	An extension package that provides domain-specific metrics, feature mappings, feature descriptors, graph templates, examples, or profiling defaults.
metric descriptor registry	A registry describing raw metrics and how they can be interpreted, aggregated, and mapped to canonical features.
graph declaration	The part of a manifest that declares graph nodes, graph edges, influence roles, sensitivities, confidence, and applicability conditions.
profiling policy declaration	The part of a manifest that declares default observation strategies, required observation points, sampling rules, and profiling overhead limits.
schema	A machine-validation structure for manifests, descriptors, registries, domain packs, or graph declarations.
versioning	The practice of assigning versions to schemas, manifests, domain packs, task kinds, feature registries, or graph structures in order to manage evolution and compatibility.
compatibility	The ability of manifests, schemas, domain packs, task versions, and graph structures to work together without semantic or validation conflicts.
coverage	The set of task kinds, domains, features, metrics, observation points, or risks covered by a model, manifest, or domain pack. Coverage is not the same as global completeness.
known gap	A missing, unavailable, weak, or intentionally unsupported feature, observation, domain, or graph relationship. Known gaps should remain visible rather than being hidden inside the model.
residual visibility	The ability of the model to expose unexplained behavior through residual error, reduced confidence, or missing-feature indicators.
canonical namespace	A stable naming scheme for canonical features, resource dimensions, occupancy dimensions, and risk dimensions, such as `context.network.instability` or `occupancy.worker.slot_time`.

A.11 Queue-Worker Lifecycle Terms

Table 13: Queue-worker lifecycle terms.

Term	Definition
enqueue	The lifecycle point at which a task instance is submitted to a queue.
dequeue	The lifecycle point at which a worker takes a task instance from a queue for processing.
reserve	The lifecycle point at which a worker or worker framework claims a task instance, often through a lease, visibility timeout, or reservation mechanism.
start	The lifecycle point at which a task attempt begins execution.
input fetch	The stage in which the worker fetches external input, payload data, referenced objects, or domain state required by the task attempt.
deserialize	The stage in which the worker decodes, parses, or prepares the task payload or input data for processing.
processing stage	A logical phase inside task execution, such as validation, transformation, dependency call, write, commit, or cleanup.
dependency call	An interaction with an external service, API, database, storage system, broker, or remote dependency during task execution.
retry	A repeated execution attempt or repeated operation after failure, timeout, lease expiration, cancellation, or retryable error.
backoff	A delay before retrying a task attempt or operation. Backoff can contribute to worker occupancy if execution capacity remains held.
output write	The stage in which task output, state changes, generated data, or side effects are written.
commit	The stage in which task effects are finalized, persisted, checkpointed, or made visible.
acknowledge (ack)	The lifecycle point at which a worker confirms task completion to the queue or broker.
discard	The lifecycle outcome in which a task instance or attempt is intentionally dropped or ignored.
dead-letter	The lifecycle outcome in which a task is moved to a dead-letter queue or equivalent failure storage.
finish	The lifecycle point at which a task attempt ends, whether by success, failure, timeout, cancellation, or another terminal outcome.

Term	Definition
lease expiration	The lifecycle event in which a task reservation expires before successful acknowledgement or completion.
cancellation	The lifecycle event in which a task attempt is explicitly stopped before normal completion.

B Extended Task Taxonomy

C Manifest Schema

D Algorithm Details

E Case Study Details

F Evaluation Methodology Details

G Reproducibility Checklist